CUADERNO DEL ALUMNADO

Lectura y escritura de ficheros XML con Java

DOM + JACKSON XML + CONVERSIÓN A CSV

Acceso a datos · 2.º de DAM · RA1

Nombre y apellidos: _____ Grupo: _____

En estas dos actividades desarrollarás aplicaciones que recuperan información de ficheros XML, permiten realizar un cambio y guardan el resultado. En la primera trabajarás con el árbol de nodos de DOM. En la segunda utilizarás objetos Java y Jackson XML, y convertirás los datos a un fichero CSV.

Los ejercicios son independientes. Cada uno incluye su propio fichero de partida y puede realizarse sin haber terminado el otro.

Criterios de evaluación

Criterio	Redacción	Ejercicios
RA1.c	c) Se han utilizado clases para recuperar información almacenada en ficheros.	1 y 2
RA1.d	d) Se han utilizado clases para almacenar información en ficheros.	1 y 2
RA1.e	e) Se han utilizado clases para realizar conversiones entre diferentes formatos de ficheros.	2

Indicaciones comunes

- Utiliza Java 17 o superior. Puedes consultar las dos webs de clase y adaptar los ejemplos trabajados.
- Organiza cada ejercicio en su propia carpeta e incluye dentro una carpeta `datos` para los ficheros. Utiliza rutas relativas.
- Guarda los ficheros de texto en UTF-8 para conservar correctamente las tildes y otros caracteres.
- Los datos que muestre el programa deben proceder de los ficheros. No escribas los registros directamente en el código Java para simular la lectura.
- Conserva las entradas originales. Los cambios se guardarán en los ficheros de salida indicados.
- Si no se puede leer la entrada porque falta el fichero o el XML está mal formado, muestra un mensaje comprensible y termina sin generar las salidas.
- Incluye las comprobaciones solicitadas. Basta con registrar la entrada utilizada, el resultado esperado y el resultado obtenido, junto con las capturas o salidas de consola necesarias.

Organización de la entrega final

Entrega una carpeta para cada ejercicio, con sus fuentes y su carpeta `datos`. En el ejercicio de Jackson incluye también `pom.xml`. No incluyas la carpeta `target` ni las dependencias descargadas.

Añade un único `README.md` que explique cómo ejecutar ambos programas y recoja las comprobaciones y respuestas solicitadas. Los ficheros entregados deben permitir reproducir una ejecución correcta de cada aplicación.

Ejercicio 1 / Catálogo de películas con DOM

Situación

Una biblioteca de aula presta películas y guarda su catálogo en un fichero XML. Necesita consultar las películas disponibles y actualizar el número de copias de una de ellas.

Desarrolla un programa de consola que lea el catálogo con DOM, muestre sus datos, localice una película por su identificador y permita cambiar sus copias. Después deberá guardar el catálogo completo en otro XML y volver a leerlo para comprobar el cambio.

Fichero de partida

Crea el fichero `datos/peliculas.xml` con este contenido:

```
<?xml version="1.0" encoding="UTF-8"?>
<peliculas>
    <pelicula id="P01">
        <titulo>Coco</titulo>
        <genero>Animación</genero>
        <copias>3</copias>
    </pelicula>
    <pelicula id="P02">
        <titulo>La isla del tesoro</titulo>
        <genero>Aventura</genero>
        <copias>2</copias>
    </pelicula>
    <pelicula id="P03">
        <titulo>Viaje a la Luna</titulo>
        <genero>Ciencia ficción</genero>
        <copias>0</copias>
    </pelicula>
    <pelicula id="P04">
        <titulo>El gran hotel</titulo>
        <genero>Comedia</genero>
        <copias>1</copias>
    </pelicula>
</peliculas>
```

El identificador `id` es un atributo de `pelicula`. El título, el género y las copias son elementos hijos. Esta diferencia determina cómo debes recuperar cada dato con DOM.

Trabajo que debes realizar

1. Leer y mostrar el catálogo

Carga `datos/peliculas.xml` en un `Document`. Recorre los elementos `pelicula` y muestra, para cada uno, su identificador, título, género y número de copias.

Al final del listado, muestra el número de películas y la suma de todas sus copias. Una película con cero copias sigue formando parte del catálogo y debe aparecer.

Con el fichero de partida, la salida debe contener esta información:

```
P01 | Coco | Animación | 3 copias
P02 | La isla del tesoro | Aventura | 2 copias
P03 | Viaje a la Luna | Ciencia ficción | 0 copias
P04 | El gran hotel | Comedia | 1 copia
Películas registradas: 4
Copias totales: 6
```

Puedes elegir la presentación de los mensajes, siempre que se muestren todos los datos y los totales correctos.

2. Buscar una película y actualizar sus copias

Solicita por teclado el identificador de una película. Elimina los espacios al principio y al final de la entrada y compara el identificador de forma exacta: por ejemplo, `P02`.

- Si existe, muestra todos sus datos y pide el nuevo número de copias.
- Si no existe, muestra `Película no encontrada` y termina sin crear ni sobrescribir el fichero de salida.
- La nueva cantidad debe ser un entero mayor o igual que cero. Si se introduce un número negativo o un texto como `cinco`, muestra un aviso y vuelve a pedirla.

Cuando la cantidad sea válida, cambia únicamente el contenido de `copias` de la película localizada. Muestra el título, la cantidad anterior y la nueva.

3. Guardar el catálogo actualizado

Utiliza `Transformer` para guardar el `Document` modificado en `datos/peliculas_actualizadas.xml`, con UTF-8 e indentación.

La salida debe conservar las cuatro películas, sus identificadores y todos los datos que no se han modificado. El fichero `peliculas.xml` original debe permanecer intacto.

4. Volver a leer la salida

Después de guardar, carga `datos/peliculas_actualizadas.xml` en un nuevo `Document` y muestra las copias de la película modificada. Calcula también la suma de copias del catálogo guardado.

Esta comprobación debe utilizar los datos recuperados del fichero de salida. Así demuestras que el cambio se ha almacenado en disco.

Comprobaciones de funcionamiento

Cada ejecución comienza leyendo el fichero original `peliculas.xml`.

Prueba	Entrada	Resultado esperado
Lectura y actualización	Identificador `P02` y nueva cantidad `5`	El listado inicial contiene 4 películas y 6 copias. Se cambia P02 de 2 a 5. La segunda lectura muestra P02 con 5 copias y un total de 9.
Cantidad incorrecta	Identificador `P01`; introduce primero `-1`, después `abc` y finalmente `0`	Se rechazan las dos primeras entradas. Se acepta cero. La salida conserva las cuatro películas y tiene 3 copias en total.
Identificador inexistente	Identificador `P99`	Se muestra el aviso y no se crea ni sobrescribe la salida.

Para comprobar el último caso, puedes conservar una salida correcta anterior y verificar que no cambia después de buscar `P99`.

Orientaciones

Puedes apoyarte en `DocumentBuilderFactory`, `DocumentBuilder`, `Document`, `Element`, `NodeList` y `Transformer`.

Recupera `id` mediante `getAttribute()`. Para los demás campos, busca sus elementos dentro de la película que estés recorriendo y recupera su texto. Convierte las copias a un número para poder sumarlas.

La modificación se realiza sobre el árbol que has leído. No necesitas crear un catálogo nuevo desde cero ni construir XML concatenando cadenas.

Entrega

- Código Java del programa.
- `datos/peliculas.xml` y una salida correcta `datos/peliculas_actualizadas.xml`.
- Evidencias de las tres comprobaciones.
- Una respuesta breve: ¿qué diferencia hay entre modificar el `Document` en memoria y guardar el resultado con `Transformer`?

Ejercicio 2 / Inventario de material con Jackson XML

Situación

Un departamento guarda el material del aula en un inventario XML. Quiere trabajar con esos datos mediante clases Java, actualizar las unidades de un material y obtener un fichero que pueda abrirse como una tabla en una hoja de cálculo.

Desarrolla un programa de consola que recupere el inventario con Jackson XML, actualice un material y guarde los cambios en otro XML. Después deberá leer ese XML actualizado y convertir su información a CSV.

Fichero de partida

Crea el fichero `datos/inventario.xml` con este contenido:

```
<?xml version="1.0" encoding="UTF-8"?>
<inventario>
    <material>
        <codigo>M01</codigo>
        <nombre>Ratón</nombre>
        <unidades>4</unidades>
    </material>
    <material>
        <codigo>M02</codigo>
        <nombre>Teclado</nombre>
        <unidades>2</unidades>
    </material>
    <material>
        <codigo>M03</codigo>
        <nombre>Pantalla</nombre>
        <unidades>0</unidades>
    </material>
    <material>
        <codigo>M04</codigo>
        <nombre>Cable HDMI</nombre>
        <unidades>3</unidades>
    </material>
</inventario>
```

En este XML, `codigo`, `nombre` y `unidades` son elementos hijos de `material`. No hay atributos. Los elementos `material` aparecen directamente dentro de `inventario`, sin una etiqueta adicional que envuelva la lista.

Trabajo que debes realizar

1. Preparar las clases y el proyecto

Utiliza el proyecto Maven y la dependencia de Jackson XML trabajados en clase. Crea estas clases:

Clase	Propiedades	Preparación necesaria
`Material`	`String codigo`, `String nombre`, `int unidades`	Constructor vacío, getters y setters. Puedes añadir un constructor con parámetros si lo necesitas.
`Inventario`	`List<Material> materiales`	Lista inicializada, constructor vacío, getter y setter.

Configura el mapeo para que la raíz sea `inventario` y cada objeto de la lista se represente mediante un elemento `material`, sin envoltorio adicional. Puedes adaptar las anotaciones del ejemplo de la web: `@JacksonXmlRootElement`, `@JacksonXmlProperty` y `@JacksonXmlElementWrapper(useWrapping = false)`.

2. Leer y mostrar el inventario

Utiliza `XmlMapper` y `readValue()` para recuperar un objeto `Inventario` desde `datos/inventario.xml`.

Recorre su lista y muestra el código, el nombre y las unidades de todos los materiales. Indica también el número de materiales y la suma de sus unidades. Con los datos originales deben aparecer 4 materiales y 9 unidades.

El material `M03` debe mostrarse aunque tenga cero unidades.

3. Buscar un material y modificar sus unidades

Pide un código por teclado, elimina sus espacios exteriores y busca una coincidencia exacta en la lista.

- Si existe, muestra sus datos y solicita la nueva cantidad de unidades.
- Si no existe, muestra `Material no encontrado` y termina sin crear ni sobrescribir los ficheros de salida.
- Acepta únicamente enteros mayores o iguales que cero. Si la cantidad es incorrecta, informa y vuelve a pedirla.

Modifica las unidades del objeto encontrado mediante su setter. Muestra su nombre, las unidades anteriores y las nuevas.

4. Guardar y recuperar el inventario actualizado

Utiliza `writeValue()` para guardar el objeto `Inventario` completo en `datos/inventario_actualizado.xml`, con formato legible y codificación UTF-8.

La salida debe mantener la estructura del fichero de partida, los cuatro materiales y todos los campos que no se han cambiado. Conserva intacto `inventario.xml`.

Después, deserializa `inventario_actualizado.xml` en una nueva instancia de `Inventario`. Muestra las unidades del material modificado y la suma de unidades de esta nueva instancia.

5. Convertir el XML actualizado a CSV

A partir de los objetos recuperados en la segunda lectura, genera `datos/inventario.csv` en UTF-8. El fichero tendrá una cabecera y una fila por material, en el mismo orden que el XML actualizado.

Utiliza el punto y coma como separador y estos nombres de columnas:

```
codigo;nombre;unidades
```

Para escribir el CSV puedes utilizar `BufferedWriter` o `Files.newBufferedWriter()`. Cierra el escritor al terminar, preferiblemente mediante `try-with-resources`.

En esta actividad puedes dar por supuesto que los nombres no contienen punto y coma, comillas ni saltos de línea. No se pide implementar un exportador para cualquier posible contenido.

Si actualizas `M02` a `5` unidades, el CSV debe contener:

```
codigo;nombre;unidades
M01;Ratón;4
M02;Teclado;5
M03;Pantalla;0
M04;Cable HDMI;3
```

Muestra un mensaje con el número de filas de datos exportadas, sin contar la cabecera. En este caso serán cuatro.

Comprobaciones de funcionamiento

Cada ejecución comienza leyendo el fichero original `inventario.xml`.

Prueba	Entrada	Resultado esperado
Lectura, actualización y conversión	Código `M02` y nueva cantidad `5`	La lectura inicial muestra 4 materiales y 9 unidades. La segunda lectura muestra M02 con 5 y un total de 12 unidades. El CSV coincide con el ejemplo anterior.

Prueba	Entrada	Resultado esperado
Cantidad incorrecta	Código `M04`; introduce primero `-2`, después `tres` y finalmente `0`	Se rechazan las dos primeras entradas. Se acepta cero. El XML actualizado y el CSV conservan los cuatro materiales; M04 tiene 0 y el total es 6 unidades.
Código inexistente	Código `M99`	Se muestra el aviso y no se crea ni sobrescribe el XML actualizado ni el CSV.

Comprueba el CSV con un editor de texto: debe tener cinco líneas, conservar las tildes y mantener los valores cero. Si existe una salida anterior, verifica que la prueba con `M99` no la modifica.

Orientaciones

El proceso es: leer XML → recuperar objetos → modificar un objeto → guardar XML → volver a leer XML → escribir CSV.

Guarda la colección completa después de realizar el cambio. Si guardas únicamente el material modificado, perderás los demás registros en el fichero de salida.

La conversión a CSV cambia la representación de la información: los elementos XML pasan a ser filas y columnas de texto. En este apartado se comprueba el criterio e).

Entrega

- Proyecto Maven con las clases `Material`, `Inventario` y el programa principal, junto con `pom.xml`.
- `datos/inventario.xml`, una salida correcta `datos/inventario_actualizado.xml` y su correspondiente `datos/inventario.csv`.
- Evidencias de las tres comprobaciones.
- Respuestas breves a estas preguntas: ¿qué operaciones realizan `readValue()` y `writeValue()`? ¿Por qué la exportación a CSV es una conversión de formato?